

1 Question

Write a method named *stitching* that accepts two integer parameters w and h and that returns a two dimensional array of numbers. Each of the h rows will contain w integers. The first number on each row is the row number.

```
stitching(6, 2);    stitching(2, 3);  
1 2 3 4 5 6        1 2  
2 3 4 5 6 7        2 3  
                  3 4
```

2 Question

Write a method named *evenBeforeOdd* that takes an array of integers as a parameter and rearranges its elements so that all even values appear before all odds. For example, if the following array is passed to your method:

5, 2, 4, 9, 3, 6, 2, 1, 11, 1, 10, 4

Then after the method has been called, one acceptable ordering of the elements would be:

4, 2, 4, 10, 2, 6, 3, 1, 11, 1, 9, 5

The exact order of the elements does not matter, so long as all even values appear before all odd values. For example, the following would also be an acceptable ordering:

2, 2, 4, 4, 6, 10, 1, 1, 3, 5, 9, 11

Hint: Look for elements that are at inappropriate places in the array and move them to better locations.

3 Question

Write a recursive method

```
int S(int N, int k)
```

to calculate Stirling numbers of the first kind where

$$S(n, n) = 1$$

$$S(n, 0) = 0, \text{ if } n \neq 0$$

$$S(n, 0) = 1, \text{ if } n = 0$$

$$S(n, k) = kS(n-1, k) + S(n-1, k-1)$$

4 Question

Suppose that we are modelling AmiralBatti game. Write the Java method **int numberOfHits(boolean[][] shots, boolean[][] shipParts)**, which counts the number of ship parts that are hit. In this function; `shots[i][j]` is true, if we have shot the cell in the *i*'th row and *j*'th column; false otherwise. Yet in this function; `shipParts[i][j]` is true, if there is a ship part in the cell in the *i*'th row and *j*'th column.

<i>shots</i> =	true	true	false	false
	false	false	false	false
	true	true	false	false
	false	false	false	false

<i>shipParts</i> =	true	false	true	true
	false	false	false	false
	false	true	true	true
	false	false	false	false

numberOfHits returns 2.

5 Question

Write a method that reads N points (p_x, p_y) in the 2- d euclidian space and returns the number of points whose distance to the center those N points is greater than M . Center of N points (m_x, m_y) is calculated as:

$$m_x = \frac{\sum_{i=1}^N p_x}{N}$$

.

$$m_y = \frac{\sum_{i=1}^N p_y}{N}$$

. The distance between two points p and q in 2- d space is calculated as

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$$

Your method should use the following methods

- a method to calculate the distance between two points
- a method to calculate the center of N points

which must also be implemented.

6 Question

Write a Java method **sortAlternate**, which takes a 2d array *A* as argument and sorts the rows of the arrays alternately. First row is sorted in increasing order, second row in decreasing order, third row in increasing order, etc. Write helper methods

sortIncreasingOrder(int[][] *A*, int *rowNo*)

sortDecreasingOrder(int[][] *A*, int *rowNo*),

which sorts the row of the 2d-array *A* with index *rowNo* in increasing and decreasing order respectively. Then use those helper methods in **sortAlternate**.

$$A =$$

3	7	12	1
1	8	4	6
6	14	0	2

$$A' =$$

1	3	7	12
8	6	4	1
0	2	6	14